

Threat Model Project

Student Project

Web Age Solutions

Threat Model Project

This project for developers offers an educational opportunity to apply security principles in an MERN (MongoDB, Express.js, React, Node.js) stack application. Aligned with the security fundamentals covered in the training program, the project emphasizes the implementation of essential security measures.

Tasks

Develop a threat model according to Travelers standards for your project. Regardless of whether you implement the full application or not, address areas as discussed in class in the STRIDE model.

- Prepare threat modeling
- Describe the data flow
- Applying secure design principles
- Optionally integrate vulnerability management and scanning tools
- Present on your findings and actions on the last day of the project

You can implement user authentication, authorization, secure password or secrets storage, multi-factor authentication, or any of the controls and mitigation we've discussed in the course. As many working parts of the application as possible are desired but the application need not be 100% complete. It should show efficient use of the project time allotted. For some controls, a plan or pseudo code may be all that can be implemented during the allocated project timeframe.

The App

MERN (MongoDB, Express.js, React, Node.js) stack is a popular choice for building web applications. However, like any other technology stack, MERN apps are susceptible to various security concerns. Review the example project provided within the LabFiles folder.

cd C:\Labfiles\ExampleProject

Start by getting each part of the application to run with the React code supplied. Additional software to utilize in the project is found under the [C:\Software](#) folder. Ensure you start with the versions of software provided.

cd C:\Software

MongoDB

1. Install MongoDB.
 - The file mongodb-windows-x86_64-6.0.5-signed.msi is located under the [C:/Software](#) folder.



2. Start MongoDB. It is started under the path you installed the server software. For instance: /MongoDB/server/6.0/bin folder

mongod

3. Load data into MongoDB.
 - mongoimport is located under the MongoTools folder in C:\Software
 - The SWAPI data is found @ <https://github.com/olaekdahl/swapi/tree/master/json-data>.
 - You'll load at minimum *characters*, *films*, *planets*, *films_characters*, and *films_planet* from where you unpack it.
 - Example **C:\swapi-data**

Example: **mongoimport --uri mongodb://localhost:27017/swapi --collection films --drop --file c:/swapi-data/films.json --jsonArray**

4. Using mongosh, examine the data added and make sure it loaded as expected.

mongosh

```

C:\> mongosh mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Users\wasadmin> mongosh
Current Mongosh Log ID: 65f9a506d6e28d825f8bf201
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&
.2.0
Using MongoDB:      6.0.5
Using Mongosh:      2.2.0

For mongosh info see: https://docs.mongodb.com/mongodb-shell/

-----
  The server generated these startup warnings when booting
  2024-03-18T13:44:23.056-04:00: Access control is not enabled for the database. Read and write access
  igation is unrestricted
  2024-03-18T13:44:23.062-04:00: This server is bound to localhost. Remote systems will be unable to co
  nnect to this server. Start the server with --bind_ip <address> to specify which IP addresses it should serve responses
  on. To bind to all interfaces, use --bind_ip_all. If this behavior is desired, start the server with --bind_ip 127.0.0.1
  to bind to all interfaces. If this behavior is desired, start the server with --bind_ip 127.0.0.1
  this warning
-----

test> use swapi
switched to db swapi
swapi> show tables
characters
films
planets

```

5. Navigate to the swapi db and review the tables were added. For example:

use swapi

show tables

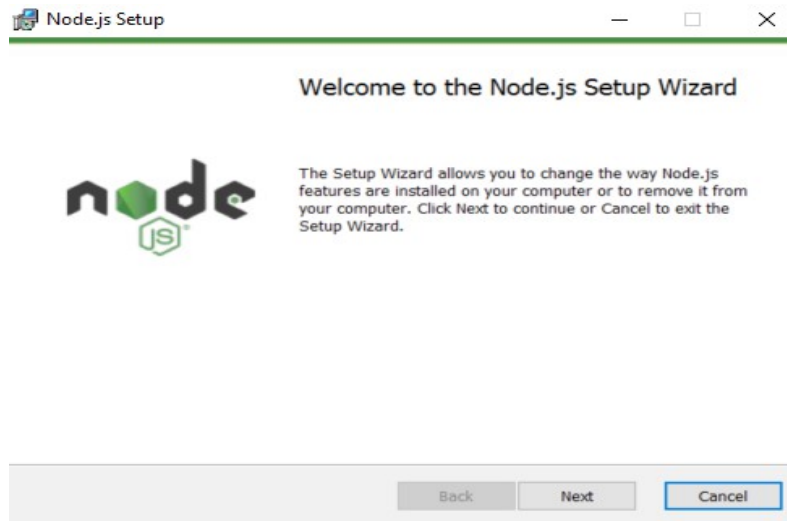
db.characters.find()

6. Once you satisfied you have all five tables properly prepared move onto Node.

Node

1. Install the node version software from under the [C:\Software](#) folder

node-v16.15.1-x64.msi



Express

1. Dependencies are already installed. Run.

npm install

React

1. Review the my-app and server folders under the ExampleProject folder.
2. Run npm start and review any missing dependencies. Fix as appropriate for the app to start on <http://localhost:3000>

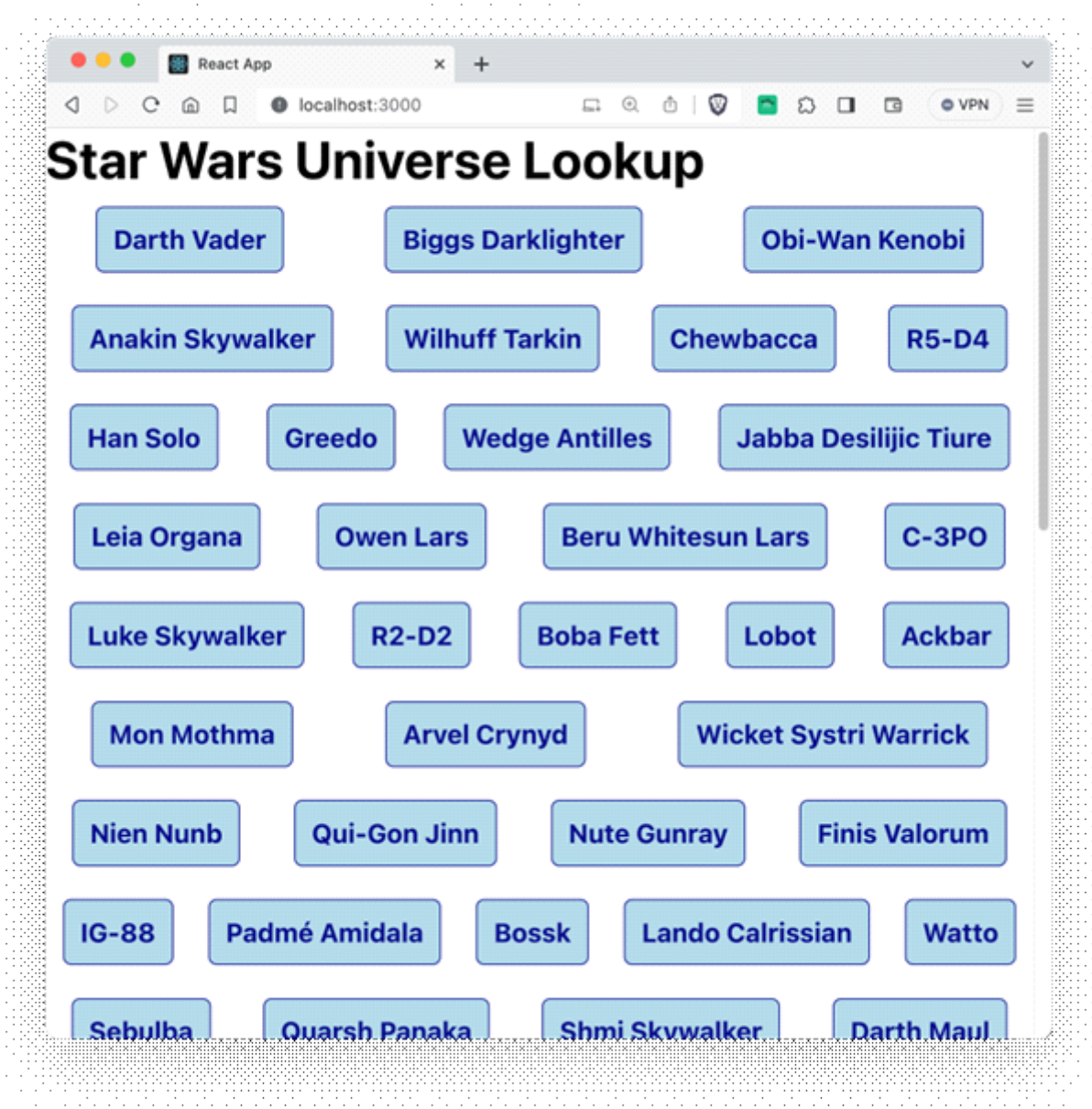
npm start

3. Postman is available for troubleshooting application issues. You can install it from <C:/Software>

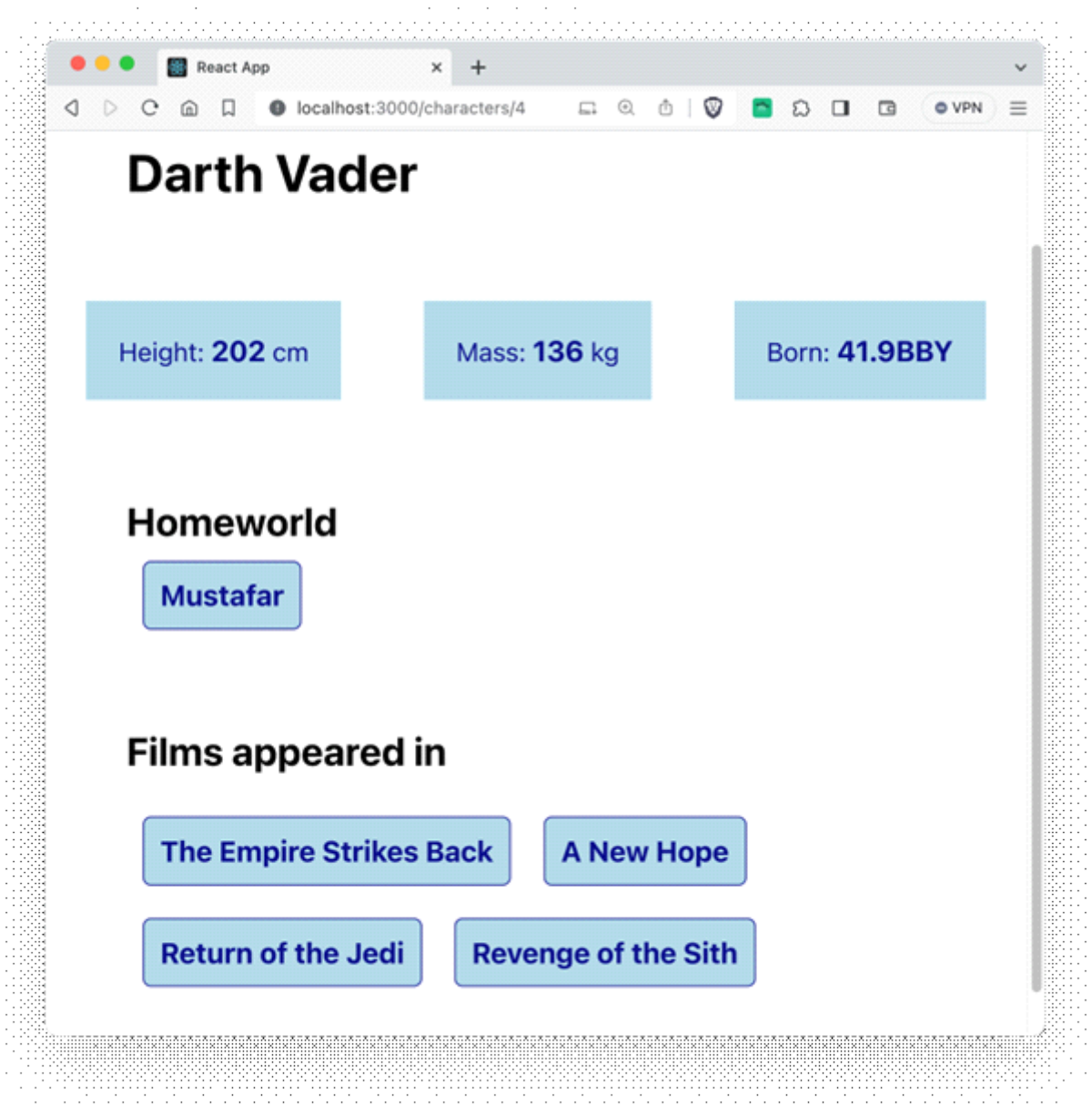
Note: You may need to install missing libraries for the React app to run.

The Resulting Site

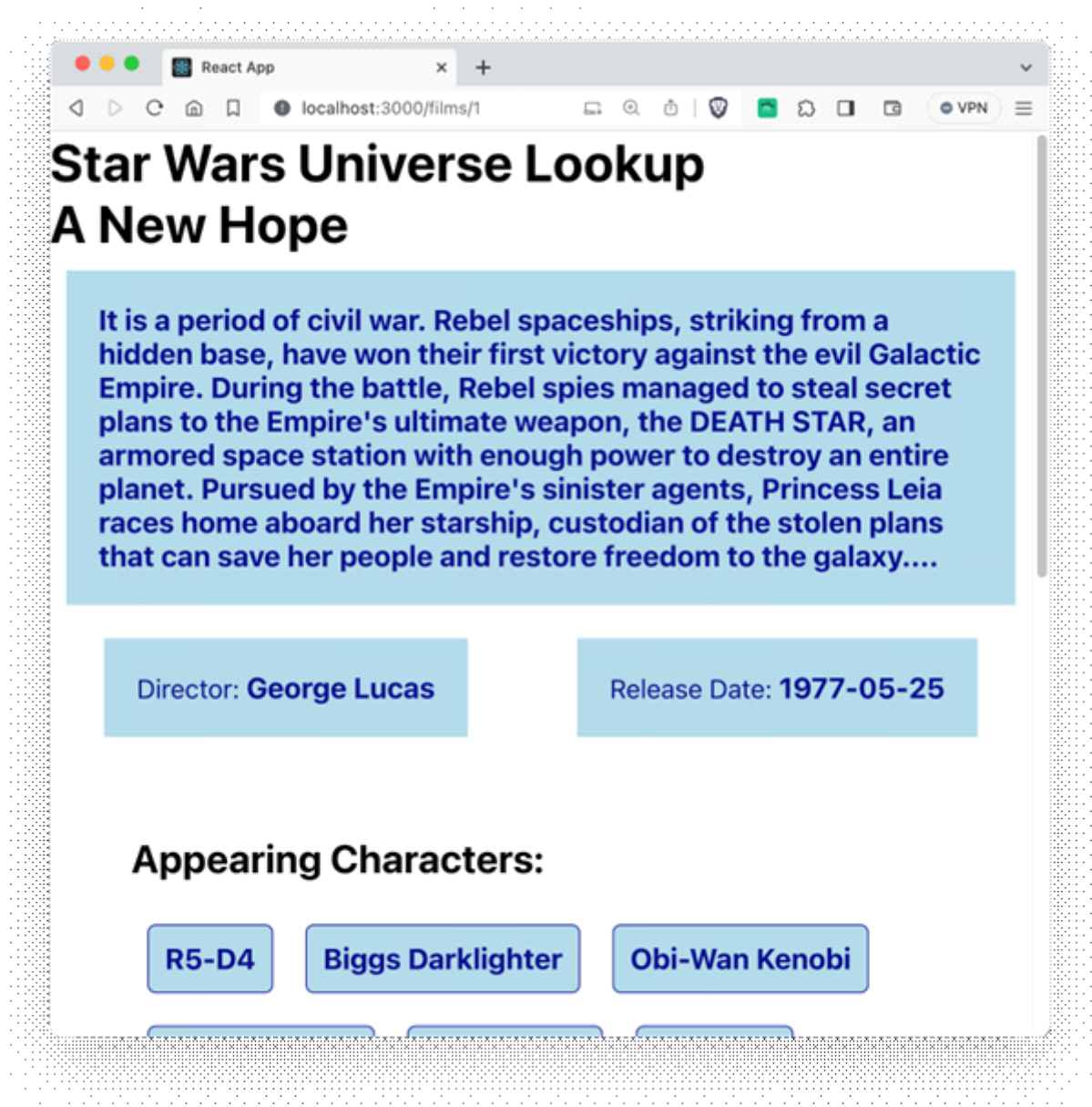
The result will incorporate all for parts of MERN and utilize the Star Wars API Database installed locally.



Clicking on a name will route you to the details component for that character. (Note that you're using a route parameter here). Each character has a home world and a list of films they've appeared in.



You'll be able to click on film and navigate to the details component for that film. The film will have a list of characters and a list of planets. These also are clickable so the user can get details for them.



As you can see, you can navigate between characters, planets, and films.

Developing a Threat Model

STRIDE model

To better help you formulate the kinds of security issues to be identified, use the STRIDE categorizes which show different types of threats.

Category	Description
----------	-------------

Spoofing	Involves illegally accessing and then using another user's authentication information, such as username and password
Tampering	Involves the malicious modification of data. Examples include unauthorized changes made to persistent data, such as that held in a database, and the alteration of data as it flows between two computers over an open network, such as the Internet
Repudiation	Associated with users who deny performing an action without other parties having any way to prove otherwise—for example, a user performs an illegal operation in a system that lacks the ability to trace the prohibited operations. Non-Repudiation refers to the ability of a system to counter repudiation threats. For example, a user who purchases an item might have to sign for the item upon receipt. The vendor can then use the signed receipt as evidence that the user did receive the package
Information Disclosure	Involves the exposure of information to individuals who are not supposed to have access to it—for example, the ability of users to read a file that they were not granted access to, or the ability of an intruder to read data in transit between two computers
Denial of Service	Denial of service (DoS) attacks deny service to valid users—for example, by making a Web server temporarily unavailable or unusable. You must protect against certain types of DoS threats simply to improve system availability and reliability
Elevation of Privilege	An unprivileged user gains privileged access and thereby has sufficient access to compromise or destroy the entire system. Elevation of privilege threats include those situations in which an attacker has effectively penetrated all system defenses and become part of the trusted system itself, a dangerous situation indeed

1. Source: <https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats>

Stride replaced DREAD at Microsoft in 2008.

DREAD is part of a system for risk-assessing computer security threats that was formerly used at Microsoft.[2] It provides a mnemonic for risk rating security threats using five categories.

The categories are:

Damage – how bad would an attack be?

Reproducibility – how easy is it to reproduce the attack?

Exploitability – how much work is it to launch the attack?

Affected users – how many people will be impacted?

Discoverability – how easy is it to discover the threat?

2. Source: [https://en.wikipedia.org/wiki/DREAD_\(risk_assessment_model\)](https://en.wikipedia.org/wiki/DREAD_(risk_assessment_model))

MERN App security issues

Here are the top 10 security concerns with MERN apps according to an AI (ChatGPT :-)):

1. **Injection Attacks:** MongoDB and Express.js applications are vulnerable to injection attacks such as SQL injection and NoSQL injection.
2. **Cross-Site Scripting (XSS):** React applications are prone to XSS attacks if user input is not properly sanitized.
3. **Cross-Site Request Forgery (CSRF):** CSRF attacks can be launched against MERN apps to execute unauthorized actions on behalf of authenticated users.
4. **Insecure Authentication:** Improper implementation of authentication mechanisms in Node.js can lead to vulnerabilities such as weak password storage, session fixation, and session hijacking.
5. **Insecure Direct Object References (IDOR):** Inadequate access controls in MongoDB can result in IDOR vulnerabilities, allowing attackers to access unauthorized data by manipulating object references.
6. **Sensitive Data Exposure:** Failure to secure sensitive data such as passwords, API keys, and session tokens can lead to data breaches.
7. **Insecure Deserialization:** Node.js applications may be vulnerable to insecure deserialization attacks if user-supplied data is deserialized without proper validation.
8. **Denial of Service (DoS) Attacks:** MERN apps can be targeted by DoS attacks that aim to overwhelm the server with excessive requests, leading to service disruption.
9. **Broken Authentication:** Weak authentication mechanisms, such as predictable session IDs or lack of session expiration, can result in broken authentication vulnerabilities.
10. **Security Misconfiguration:** Misconfigurations in server settings, application frameworks, or cloud services can expose MERN apps to various security risks.

By addressing these top security concerns and adopting proactive security measures, you can strengthen the security posture of your MERN applications and protect them from potential threats and vulnerabilities.

Work to be done for Presentation:

Your team should:

- Review the above MERN security issues and select 3 – 5 of these concerns *and/or* use your own additional security concern thoughts, that should be

addressed within the current SWAPI Example Project provided.

- Make sure you reference where and how these should be addressed in the project and within the code.
- Use one or more resources to understand each vulnerability as it pertains to one or more parts of the current application. Some options include:
 - <https://security.snyk.io/vuln>
 - <https://nvd.nist.gov/vuln/search/results>
 - <https://www.cvedetails.com/vulnerability-list>
 - https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
 - etc
- Make suggestions, supply a solution to the concerns you picked.
 - You don't have to implement the code per se, but if you can... :-)
 - If you can't at least provide pseudo code that represents your planned implementation.
- Be prepared to answer questions on your suggested plan(s).

Project Presentation Guidelines:

Timing: Allocate 10 minutes to present, 5 minutes for audience Q&A.

Backup: Maintain a secondary copy of all working code, configurations, and commands.

Prepare Professional Slides:

(These are suggestions. Optimize for "aha", use Travelers logo)

1. Introduction: Team Introduction
2. Topic: Introduce your project's main idea
3. Project Overview: Explain the project's objectives and what you've accomplished.
3. Project Architecture/Design: Show system design or application architecture diagrams, outlining major components and their interactions.
4. Tools/Software: Specify the development and testing tools used and their roles in your project.

E.g., NPM, Node, React, Express, Helmet, OpenSSL, etc
5. Challenges: Narrate the problems your team faced and the solutions you implemented.

6. Learning: Compare your technical skills and knowledge before and after the project.
7. Security Vulnerabilities: Talk about the security vulnerabilities you identified in your project
8. Solutions: Discuss your vulnerability mitigation strategies and how you implemented them
9. Threat Model: Present a threat model for your project, including STRIDE / DREAD, data flow diagram(s).
Follow closely Travelers Threat model template.
10. Demo: Perform a live demonstration of your project in action. Show what you fixed. Show what you might fix if you had more time.
11. References: List any references and resources utilized.
12. Q&A: Invite audience into a Q&A session. Can you keep your audience engaged enough so they ask interesting questions?

REMINDER: The point of the presentation is to showcase that you learned and you applied what you learned in a productive way. How does your presentation do that?